Flood Forecasting Simulation

COIS 4350 — High Performance Computing

Ferdinand Nel |0821785

1. Introduction

Flood forecasting is a critical tool for protecting lives and infrastructure. Cities like Venice and Bangkok face yearly flooding, and being able to predict and plan for these events is essential. Real-world flood simulations operate on grids of 10,000x10,000 cells or larger, with multiple values per cell being recalculated continuously throughout a storm. optimization methods are necessary to make these calculations fast enough to be useful in emergency response situations.

For this project, I built a grid-based flood forecasting simulation in C, using real hydrology algorithms to simulate rainfall, soil absorption, water routing and flooding. Once the serial version was complete, I parallelized it using OpenMP, testing across multiple grid sizes and thread counts to measure performance gains. Simulation results were also validated against expected hydrological behaviour.

The simulation is built around three core components — a Digital Elevation Model representing terrain, the Green-Ampt infiltration model calculating soil absorption per timestep, and a D8-based water routing algorithm moving surface water downhill. The project went through several iterations before producing realistic results, particularly around the water routing model, which initially moved water too aggressively. A dynamic surface routing approach was eventually implemented that routes water based on combined terrain and water depth, allowing flood zones to spread naturally over time.

2. Background and Theory

2.1 Digital Elevation Models (DEM)

A DEM is a 2D grid where each cell stores an elevation value representing the height of a point on a landscape, measuring bare ground only. In hydrology, DEMs are the foundation of terrain-based simulation — water cannot be routed without knowing which direction is downhill.

I used four grid sizes: 1000x1000, 2000x2000, 5000x5000, and 8000x8000. Each cell was assigned a random elevation between 0 and 1000 metres, then smoothed using a box blur algorithm that averages each cell with its neighbours over four passes. Without smoothing, neighbouring cells could have wildly different elevations which would produce meaningless routing results.

I chose synthetic terrain over real DEM data because parsing real files would require libraries like libgeotiff or GDAL, complexity unrelated to the goals of this project. If I were to continue developing this simulation, importing a real DEM would be the first change I would make. Early in testing, I also discovered that using a clock-based random seed meant every run produced different terrain, making fair comparisons impossible. Switching to a fixed seed of 42 solved this.

2.2 D8 Flow Direction Algorithm

The D8 algorithm, developed by O'Callaghan and Mark (1984), examines all eight neighbours of each cell and routes water to the one with the lowest elevation. Results are stored in a flow direction array precomputed once after terrain setup. Sink cells — where all neighbours are higher — are assigned a value of -1 and simply accumulate water.

The biggest limitation of D8 is that water routes to only a single neighbour with no flow splitting. During early development, this caused flood zones to reach maximum extent almost instantly, regardless of storm duration, since all water rushed to the same low points within the first few timesteps. This led me to replace the precomputed flow direction array with a dynamic surface routing approach covered in Section 3.3, where routing is based on combined terrain and water depth rather than bare elevation alone.

2.3 Green-Ampt Infiltration Model

To create a more accurate simulation, I needed to properly model how rainfall is absorbed into the ground for different soil types. I used the Green-Ampt infiltration model, originally developed by Green and Ampt in 1911 and still widely used in modern hydrology today. The model predicts how water soaks into soil over time, with the key behaviour being that the infiltration rate decreases as the soil becomes more saturated. Early in a storm, dry soil absorbs water quickly. As cumulative infiltration builds up, the rate slows down significantly.

The infiltration rate at any given moment is calculated as:

$$f = Ks \times (1 + (\psi \times \Delta\theta) / F)$$

Where Ks is the saturated hydraulic conductivity — how fast water moves through the soil.

ψ is the wetting front suction head — how strongly the soil pulls water downward.

$\Delta\theta$ is the moisture deficit — a value between 0 and 1 representing how dry the soil is before the storm begins.

F is the cumulative infiltration so far, tracked per cell in the totalInfiltration array and updated every timestep.

Soil parameters were sourced from Rawls, Brakensiek and Miller (1983), which provides a standard soil texture table used across hydrology literature. The simulation supports four soil types — Clay, Loam, Sand and Silt — each with distinct parameter values. The user selects their soil type at runtime, allowing different scenarios to be tested without recompiling.

One issue I encountered early in development was a divide-by-zero error on the first timestep, since cumulative infiltration starts at zero. To fix this, each cell's initial infiltration value is set to suctionHead × moistureDeficit for the selected soil type. This value is physically meaningful — it represents the initial suction-driven infiltration capacity of the soil before any rain falls — and prevents the formula from blowing up on the first calculation

2.4 OpenMP Parallel Programming

I chose OpenMP over pthreads for ease of implementation — adding `#pragma omp parallel for` above existing loops was enough to distribute work across threads without restructuring the code. Thread count is controlled via `OMP_NUM_THREADS` in the terminal, making it easy to test 1, 2, 4, and 8 threads without recompiling.

Early in testing, I noticed parallel runs were reporting longer times than serial runs. The problem was `clock()`, which accumulates CPU time across all threads — 8 threads running for 4 real seconds reports 32 seconds. Switching to `omp_get_wtime()` fixed this by measuring actual wall clock time.

The waterRoute function required extra care. Multiple threads writing to the same neighbour cell simultaneously causes a race condition. I resolved this with `#pragma omp atomic` on every write to the shared tempWater array, ensuring correct accumulation at the cost of some contention.

Testing was done on an Apple M-series processor with unified memory architecture, where all cores share the same memory bus. This likely contributed to atomic contention in the results — traditional x86 hardware with separate cache hierarchies would probably show stronger scaling.

3. Implementation

3.1 Data Structures

I used four flat 1D arrays over 2D arrays or an array of structs to preserve cache locality — contiguous memory means fewer cache misses when threads process cells sequentially. The four arrays were elevation[] (float, fixed after setup), water[] (float, updated every timestep), totalInfiltration[] (float, updated every timestep), and flowDirection[] (int, precomputed — replaced by dynamic routing in the final version).

I chose a Struct of Arrays layout over an Array of Structs because each loop only needs one type of data. The infiltration loop, for example, only touches totalInfiltration[] — with AoS, it would have to skip over elevation, water, and flow direction data for every cell just to find it. At a million cells, the wasted bandwidth adds up.

The 2D grid is represented in the 1D array using `index = row * NUMCOLS + col`. Neighbour finding uses two offset arrays — rowOffset[] and colOffset[] — covering all 8 directions without hardcoding.

All arrays are declared globally. At 8000x8000, each float array is roughly 256MB — stack allocation at that size crashes immediately. When I attempted 10,000x10,000, the program failed due to memory mapping limits, which set the practical ceiling for single-machine testing. A SoilType struct groups the three Green-Ampt parameters together for clean function passing.

3.2 Simulation Pipeline

The simulation is divided into two separately timed phases: setup and the main simulation loop. Separating these allows for a clearer analysis of where time is actually being spent.

The setup phase generates and prepares the terrain before any storm simulation begins. It consists of three steps. First, generateElevations() fills the grid with random elevation values between 0 and 1000 metres. Second, smoothElevations() applies four passes of box blur averaging, using a temporary array to store each pass's results before copying back — this prevents already-smoothed values from influencing the current pass mid-loop. Third, calculateFlowDirection() runs the D8 algorithm across the entire grid and precomputes a flow direction for each cell based on bare terrain elevation.

Once setup is complete, the main simulation loop runs for however many hours the user specified. Each timestep represents one hour and executes three operations in order. applyRainfall() adds the user-specified rainfall amount in mm/hr to every cell's water depth. applyInfiltration() then applies the Green-Ampt formula per cell, calculating how much water soaks into the ground that hour and subtracting it from the surface water — capped at the available water depth so a cell can never infiltrate more than it has.

Finally, waterRoute() moves surface water downhill to lower neighbours based on the dynamic surface routing model described in Section 3.3.

After all timesteps complete, checkFlooding() scans every cell and marks any cell where water depth exceeds the flood threshold as flooded. All results and timing data are then written to .dat files for analysis and graphing with gnuplot.

One design decision that kept testing flexible was making the three main simulation parameters user inputs rather than compile-time constants. The user selects soil type, storm duration in hours, and rainfall intensity in mm/hr at runtime. This meant I could run different scenarios — comparing clay vs sand, or a 6-hour storm vs a 72-hour storm — without recompiling the program each time.

3.3 Water Routing Evolution

3.3 Water Routing Evolution

Getting the water routing to behave realistically was one of the most challenging parts of this project. It went through three distinct versions before producing meaningful results.

Version 1 — Simple D8 Routing

The initial implementation used the precomputed D8 flow direction array to route all of a cell's surface water to its single lowest neighbour every timestep. The problem with this approach was that water reached its final destination almost instantly. Within the first timestep, every cell that was capable of flooding had already done so. When I ran duration experiments comparing a 6-hour storm to a 72-hour storm, the percentage of flooded cells was identical. More hours of rain made the same cells deeper, but never caused new cells to flood. This was clearly not realistic behaviour and needed to be fixed.

Version 2 — Fractional Flow Rate

To slow the routing down, I introduced a flow rate fraction of 0.5, meaning only 50% of a cell's water would be routed to its lowest neighbour each timestep, with the remaining 50% staying in place. The intention was that water would build up more gradually. Unfortunately, this made things worse — the water being retained locally caused faster accumulation in place, and flood zones still plateaued almost as quickly as before. The fraction slowed the movement but didn't fix the underlying problem.

Version 3 — Dynamic Surface Routing with Ponding Depth

The real fix required rethinking the routing model entirely. I introduced a constant called PONDINGDEPTH set to 5mm. Instead of routing all water or a fixed fraction, each cell now retains water up to the ponding depth and only routes the excess above that threshold. More importantly, routing is no longer based on bare terrain elevation alone — it uses the combined water surface elevation, calculated as terrain height plus current water depth. Water is then distributed proportionally across all lower-surface neighbours weighted by the height difference, rather than routing everything to a single lowest cell.

This change had two major effects. First, depressions fill naturally — as a sink accumulates water, its surface elevation rises until it exceeds neighbouring cells, at which point water spills outward into adjacent cells. Second, flooding now spreads gradually over time rather than snapping to its final state in the first timestep. The result was a realistic temporal progression — as storm duration increases, flood zones grow steadily outward.

This was also the version where I identified and fixed the random seed issue. With different terrain generated each run, duration comparisons were meaningless because each test was running on a completely different landscape. Fixing the seed to a constant value ensured every experiment used identical terrain, making results directly comparable.

3.4 OpenMP Parallelization

Converting the simulation to use OpenMP parallelization was relatively straightforward. The bulk of the work was identifying which functions inside the simulation loop were safe to parallelize and which required special handling.

Three functions inside the main simulation loop were parallelized. applyRainfall() and applyInfiltration() are both embarrassingly parallel — each cell's calculation is completely independent of every other cell in the same timestep. Adding a single `#pragma omp parallel for` directive above each loop was all that was required.

waterRoute() required more care. The function routes each cell's water to one or more lower neighbours by writing to a shared array called tempWater. The problem is that multiple cells can route water to the same neighbour simultaneously. Without protection, two threads could both read tempWater at the same value, each add their contribution, and then both write back — with the second write overwriting the first. This is a classic race condition and would cause water to silently disappear from the simulation.

The fix was `#pragma omp atomic`, applied to every write to tempWater. This ensures only one thread at a time can execute each write operation, preventing any overwrites.

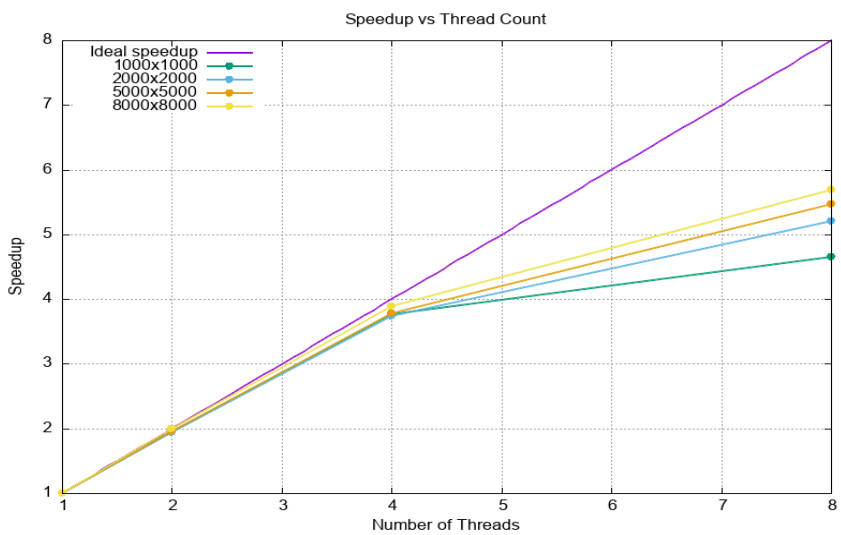
Every tempWater write in the function required the atomic directive — both the retained water written back to the current cell and the routed water written to each lower neighbour.

One additional thread safety issue was the neighbour arrays used inside waterRoute(). These were initially declared outside the parallel loop, meaning all threads shared the same array. Each thread would overwrite the other's neighbour data mid-calculation. Moving the neighbour arrays inside the parallel loop gives each thread its own private copy, which resolved the issue.

Two functions were deliberately left unparallelized — smoothElevations() and calculateFlowDirection(). Both run only once during the setup phase and are timed separately from the simulation loop. Since they do not contribute to the simulation timing experiments, parallelizing them was unnecessary for the goals of this project.

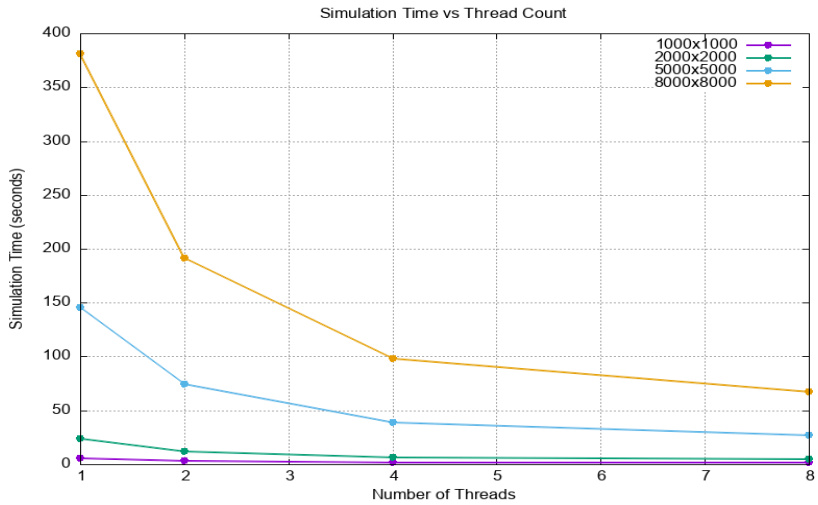
4. Results and Analysis

4.1 Performance Results



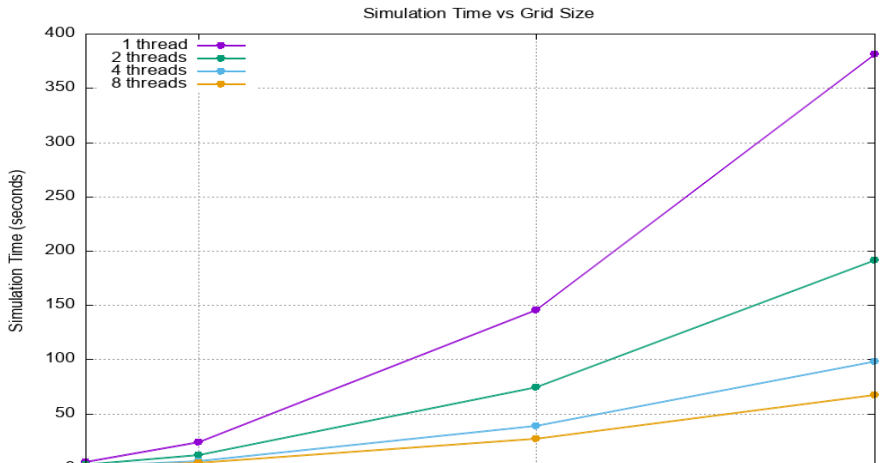
The speedup graph shows results across four grid sizes ranging from 1000x1000 to 8000x8000, tested with 1, 2, 4, and 8 threads. At 2 threads, all grid sizes achieved near-perfect 2x speedup, sitting almost directly on the ideal speedup line. At 4 threads, speedup ranged from 3.7x to 3.9x across grid sizes. At 8 threads, speedup ranged from 4.6x to 5.7x, with larger grids consistently outperforming smaller ones. Parallel efficiency at 8 threads was approximately 58-71%, which is considered good for a shared memory system that relies on atomic operations for correctness.

The scaling graph tells an equally clear story. Serial execution time grows steeply with grid size — from 5.8 seconds at 1000x1000 all the way to 381 seconds at 8000x8000. With 8 threads, that same range drops to 1.25 seconds and 67 seconds respectively. The gap between the serial and parallel lines widens significantly at larger grid sizes, demonstrating that parallelization becomes increasingly valuable as the problem scales up.

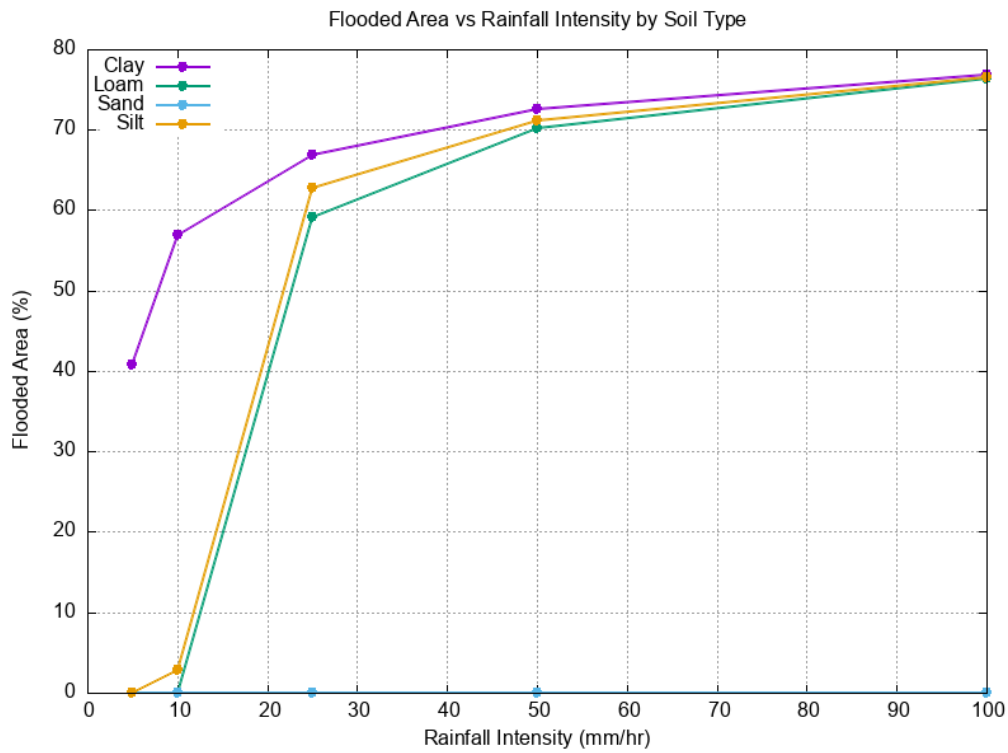


An important finding came from earlier experiments before the dynamic surface routing was implemented. With the original simple waterRoute function, adding more threads actually made the simulation slower. The reason was that each cell performed very little computation — just a single array lookup and one write. The overhead of spawning and synchronizing threads exceeded the time saved by parallel execution. Once the dynamic surface routing model was introduced, each cell required neighbour finding, surface elevation comparisons, proportional weight calculations, and multiple atomic writes. This increased the computation per cell enough that threads had meaningful work to do, which is why the final results show strong speedup. This demonstrates a key principle in high-performance computing. Parallelization is only beneficial when the computation-to-overhead ratio is high enough to justify the cost of thread management.

Finally, it is worth noting that all testing was conducted on an Apple M-series processor with a unified memory architecture. Unlike traditional x86 processors, where each core has its own cache hierarchy, all cores on Apple Silicon share the same memory bus. This increases contention on atomic operations and likely explains why speedup falls short of the theoretical ideal. Results on a traditional multi-core x86 system or cluster would likely show stronger scaling.



4.2 Simulation Results — Soil Type



The graph above shows the percentage of flooded cells for each soil type after a 24-hour storm at 25mm/hr rainfall intensity. Clay produced the highest flooding at 66.8%, which is consistent with its saturated hydraulic conductivity value of Ks = 0.3mm/hr. At that rate, clay barely absorbs any rainfall, meaning most water remains on the surface and accumulates in flood zones. Loam and silt behaved similarly, flooding at 59.2% and 62.7% respectively, reflecting their moderate infiltration capacities.

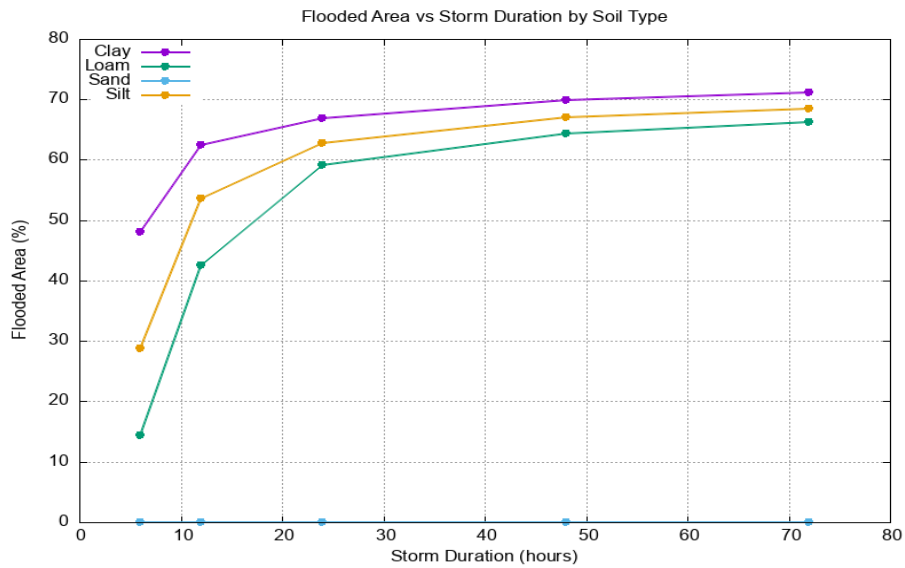
Sand produced zero flooding across the entire test. With a Ks value of 117.8mm/hr, sand absorbs rainfall nearly 5 times faster than it falls at 25mm/hr, meaning no surface water ever accumulates. The sand line sits flat at the bottom of the graph throughout.

These results are consistent with real-world soil behaviour. Impermeable soils like clay are well known to produce high surface runoff, while well-draining soils like sand rarely produce surface flooding under typical storm conditions. The fact that the simulation reproduces this ordering correctly helps validate that the Green-Ampt infiltration model is working as intended.

4.3 Simulation Results — Rainfall Intensity

As shown in the graph, clay floods at every tested rainfall intensity, producing over 40% flooding even at just 5mm/hr. Loam and silt show a clear tipping point behaviour — both sit near zero flooding at 10mm/hr before jumping to around 60% at 25mm/hr. This threshold represents the point where rainfall rate exceeds the soil's infiltration capacity and surface water begins to accumulate rapidly. Sand remains at zero flooding regardless of rainfall intensity. At extreme rainfall, all soil types except sand converge toward approximately 76%, suggesting an upper limit where terrain and routing geometry become the dominant factor rather than soil type.

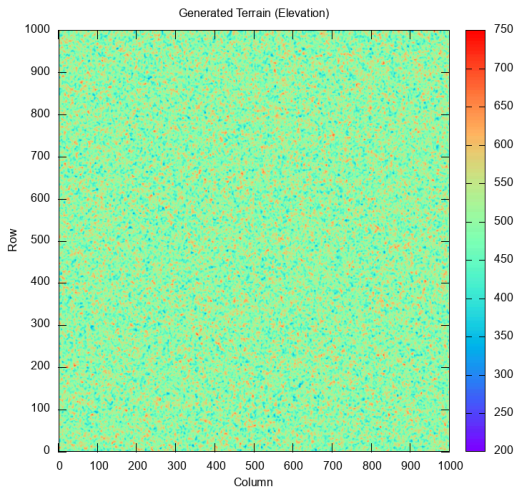
4.4 Simulation Results — Storm Duration

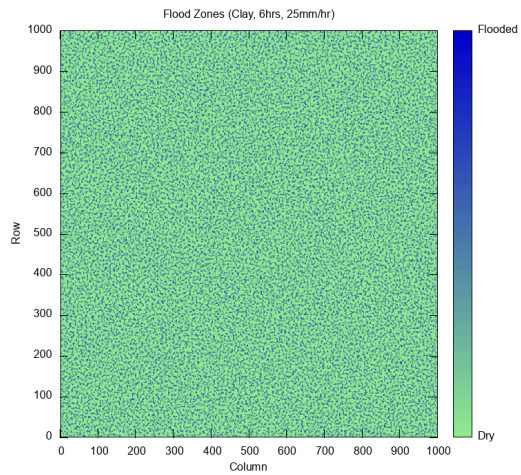


The graph above shows how flooding progresses over time for each soil type during a 25mm/hr storm. All three flooding soil types show clear temporal progression, confirming that the dynamic surface routing model produces realistic behaviour. Clay floods fastest, reaching 48% by hour 6 and 71.1% by hour 72. Loam starts slowly at 14.4% at hour 6, then accelerates as the soil saturates and infiltration capacity decreases. Silt falls between the two, flooding faster than loam but slower than clay. Sand remains at 0% regardless of storm duration.

All curves flatten as the storm progresses, reflecting the natural behaviour of flood zones reaching equilibrium as low-lying areas fill with water. Early hours see rapid flooding while later hours only produce incremental increases — consistent with real-world observations of how flood events unfold over time.

4.5 Terrain and Flood Visualizations





The two maps above show the synthetically generated terrain and the resulting flood zones after a 6-hour storm on loam soil at 25mm/hr. Elevation ranges from approximately 200 to 750 metres across the grid. Flood zones correspond to local depressions in the terrain where surface water accumulates after infiltration capacity is exceeded. The scattered flood pattern reflects the nature of randomly generated terrain — there are no connected valleys or river channels, just isolated low points distributed across the grid. With a real DEM, flood zones would appear as connected drainage networks following natural valleys and watercourses.

5. Discussion

5.1 OpenMP Analysis

The primary performance bottleneck in this simulation is the atomic operations inside waterRoute(). When multiple threads attempt to route water to the same neighbour cell simultaneously, they must queue up and wait for each other to write. This serializes the most contentious part of the parallel loop and limits how much speedup additional threads can provide. Given the routing model, this bottleneck is unavoidable —

correctness requires atomic writes, and there is no way around the contention without fundamentally changing the data structure.

A key insight from this project is that OpenMP only provides meaningful speedup when the work per thread is computationally expensive enough to justify the overhead of thread creation and synchronization. Early experiments with the simple D8 routing model showed no speedup at all — more threads actually made the simulation slower because each cell performed so little work that thread management cost more than it saved. Once the dynamic surface routing model was introduced, the computation per cell increased substantially, and OpenMP finally produced strong results. This demonstrates that parallelization is not a universal solution — it must be matched to problems with sufficient computational complexity.

5.2 Limitations and Future Work

Several improvements would make this simulation more realistic. The most impactful would be replacing synthetic terrain with real DEM data from the USGS National Map, which would produce connected drainage networks rather than scattered flood zones. Currently, a single soil type is applied uniformly across the grid — real landscapes have spatially varying soil composition. Rainfall is also constant throughout each storm, where a time-varying input would better reflect how real storms behave. The closed grid boundary inflates flooding percentages since water cannot exit — adding boundary outflow would address this.

For larger-scale simulations, MPI would be the natural next step. OpenMP is limited to shared memory on a single machine, while MPI can distribute the workload across a cluster for grid sizes that exceed single-machine memory. This was not implemented due to time constraints — OpenMP was sufficient for the scales tested here. The grid operations are also highly parallel in nature, making this simulation a strong candidate for GPU acceleration via CUDA.

6. Conclusion

This project successfully implemented a parallelized flood forecasting simulation in C using OpenMP. The Green-Ampt infiltration model and dynamic surface routing algorithm together produce physically realistic flooding behaviour across different soil types, rainfall intensities, and storm durations. At 8 threads on an 8000x8000 grid, the simulation achieved 5.7x speedup over the serial baseline, demonstrating the value of parallelization at scale.

A key finding was that parallelization only produces meaningful speedup when the computation per cell is sufficiently complex. Early versions of the routing model showed no benefit from OpenMP — it was only after introducing the dynamic surface routing model that threads had enough work to justify their overhead.

The simulation results validated the physics of the model — clay flooded most severely due to low hydraulic conductivity, sand produced no flooding at any tested intensity, and all soil types showed increasing flood area with storm duration. These results confirm that high-performance computing methods are essential for accurate and timely flood forecasting at real-world scales.

7. References

- O'Callaghan, J.F. and Mark, D.M. (1984). The extraction of drainage networks from digital elevation data. Computer Vision, Graphics and Image Processing, 28, 323-344.
- Green, W.H. and Ampt, G.A. (1911). Studies on soil physics, part I: the flow of air and water through soils. Journal of Agricultural Science, 4, 1-24.
- Rawls, W.J., Brakensiek, D.L. and Miller, N. (1983). Green-Ampt infiltration parameters from soil data. Journal of Hydraulic Engineering, 109(1), 62-70.
- OpenMP Architecture Review Board (2021). OpenMP Application Programming Interface, Version 5.1. https://www.openmp.org
- USGS National Map Downloader. https://apps.nationalmap.gov/downloader/
- OpenTopography. https://opentopography.org
- ArcGIS Pro Documentation — How Flow Direction Works. https://pro.arcgis.com/en/pro-app/latest/tool-reference/spatial-analyst/how-flow-direction-works.htm
- Oklahoma State University Open Textbook — Green-Ampt Infiltration Model. https://open.library.okstate.edu/rainorshine/chapter/6-2-green-ampt-infiltration-model/